Kathmandu University
Department of Computer Science and Engineering
Dhulikhel, Kavre

A Project Report
on
"MINI-BANKING"

[Code No.: COMP 206]

(For partial fulfillment of Year II / Semester I in Computer Engineering)

Submitted by

Safal Bastola (037960-24)

Roll no.:06

Submitted to

Sagar Acharya

Department of Computer Science and Engineering

February 24, 2026

Bona fide Certificate

This project work on

"MINI-BANKING"

is the bona fide work of

"

Safal Bastola (037960-24)

"

who carried out the project work under my supervision.

Project Supervisor

Sagar Acharya
Lecturer
Department of Computer Science and Engineering

Acknowledgements

I express my sincere gratitude to all those who supported me throughout the completion of this project. First and foremost, I am truly grateful to my computer lecturer **Mr.Sagar Acharya**, for his valuable guidance, continuous support, and encouragement throughout the entire course of this project. His insights and feedback were really helpful to improve and develop the project. I also thank the "**Department of Computer science and engineering**" for providing the necessary resources and a supportive environment to carry out this work. My heartfelt thanks goes to my friends and classmates for their collaboration and motivation, and to my family for their patience, understanding, and unwavering support. Finally, I appreciate everyone who directly or indirectly contributed to the successful completion of this project.

Abstract

Banking systems are something we interact with almost daily, yet understanding how they manage thousands of accounts efficiently isn't always clear to students learning data structures. The purpose of this mini project was to develop a Mini Banking System using C++ that demonstrates how Binary Search Trees (BST) can be applied to solve real-world problems like account management.

The system handles essential banking operations including account creation, deposits, withdrawals, account deletion, and viewing account details. All accounts are organized using a Binary Search Tree data structure, which provides efficient searching, insertion, and deletion operations. The visualizer was developed using object-oriented programming principles with a modular design, separating the Account class, BST class, and user interface into distinct components.

One of the key features I implemented was data persistence through file handling, ensuring that all account information is saved when the program closes and automatically loaded when it restarts. This makes the system practical for real-world use rather than just a theoretical exercise.

The final result is a functional console-based banking application that demonstrates the practical importance of choosing the right data structure. Through this project, I gained hands-on experience with BST operations, recursive algorithms, file I/O, and creating user-friendly interfaces. This project serves as a practical learning tool for understanding how data structures are used in actual applications and can be further enhanced by implementing self-balancing trees, adding security features, or developing a graphical user interface.

Keywords: *Binary Search Tree, Banking System, Data Structures, Account Management, File Handling, C++ Programming*

Contents

List of Figures

List of Tables

Abbreviations

BST Binary Search Tree

C++ C Plus Plus (Programming Language)

DSA Data Structures and Algorithms

GCC GNU Compiler Collection

GUI Graphical User Interface

I/O Input/Output

IDE Integrated Development Environment

O(n) Order of n (Time Complexity Notation)

OOP Object-Oriented Programming

UI User Interface

Chapter 1

Introduction

1.1 Background

Banking is something we all interact with regularly, whether it's checking our balance or making a withdrawal. For this mini project, I wanted to create a simple banking system that could handle basic operations like creating accounts, depositing money, and withdrawing funds. But instead of just making it work, I wanted to make it efficient.

That's where Binary Search Trees came in. I learned about BSTs in class and thought they'd be perfect for this project because they make searching for accounts really fast. Instead of checking every single account one by one, the BST lets me find accounts much quicker by organizing them in a smart way.

The whole system runs in the console, which keeps things simple and lets me focus on getting the core functionality right. I also added file handling so that when you close the program, all the account data gets saved and loads back up when you restart it.

Working on this project helped me understand how the data structures we learn in class actually get used in real applications. It's one thing to study BSTs on paper, but implementing one for an actual banking system made everything click for me.

1.2 Objectives

- To build a functional banking system that handles account creation, deposits, withdrawals, and account management using C++ programming.
- To implement and understand Binary Search Tree data structure in a practical application, demonstrating efficient searching, insertion, and deletion operations.
- To implement file handling for data persistence, ensuring account information is saved and loaded between program sessions.
- To apply object-oriented programming principles by organizing the system into modular classes with clear separation of concerns.

1.3 Motivation and Significance

The main motivation behind this project was simple: I wanted to bridge the gap between theory and practice. I could explain Binary Search Trees on paper, but I wanted to build something real with them. Banking seemed perfect because everyone understands what banks do, yet most don't think about how the system finds your account among thousands in just seconds.

I also wanted to create something that actually works, not just a toy program. This meant implementing file handling so data persists between sessions, making it feel like a real application.

The significance of this project is that it demonstrates why choosing the right data structure matters. With BST, searching stays fast even as accounts grow. It also shows how object-oriented programming helps organize complex systems by separating concerns into different classes. Most importantly, this project taught me that data structures aren't just academic concepts - they're practical tools .

Chapter 2

Related Works

While working on this project, I looked at several similar implementations to understand different approaches to banking systems and BST applications.

2.1 **Banking Management Systems**

Most student banking projects use simple file handling with arrays or vectors to store account data. While these work fine for small-scale projects, they don't scale well. Searching through an array of 1000 accounts means checking up to 1000 entries in the worst case. My BST approach reduces this to about 10 comparisons on average.

Some projects used linked lists, which are better for insertion and deletion but still require linear search time. Others used hash tables, which are faster for searching (O(1) vs O(log n)), but don't maintain sorted order. Since I wanted to display accounts sorted by account number, BST was the better choice.

2.2 Binary Search Tree Applications

BST is commonly used in dictionary systems, file systems, and database indexing. What I learned is that BST works best when you need both fast searching and sorted traversal. The inorder traversal property, which gives sorted output, is something I used extensively in my display functions.

One limitation of basic BST is that it can become unbalanced if data is inserted in sorted order. Self-balancing trees like AVL or Red-Black trees solve this, but I stuck with basic BST to focus on understanding fundamental concepts first.

2.3 File Handling Approaches

Different projects handle data persistence differently. Some use binary files (more efficient but harder to debug), others use databases like SQLite. I chose plain text files because they're simple, human-readable, and sufficient for this project’s scope.

Design and Implementation

I designed this system with three modular components: the Account module handles individual account data and operations (deposits, withdrawals), the BST module manages all accounts using Binary Search Tree for efficient searching and sorting, and the main program provides the user interface. This separation made development easier - I could modify each part independently without affecting the others.

System Overview

The system is composed of several major components:

- User Interface (Console): Provides an interactive menu for banking operations with formatted output and input validation.
- Account Module: Handles individual account operations and encapsulates account data (number, name, balance) with deposit and withdrawal methods.
- BST Module: Manages all accounts using Binary Search Tree, implementing insertion, deletion, and searching for efficient account lookup.
- File Handling System: Saves and loads account data to ensure persistence between sessions.

The components interact modularly - the main program calls BST methods, which interact with Account objects. This design ensures flexibility, allowing me to modify other components without affecting others.

3.1 System Requirement Specifications

3.1.1 Software Requirements

- Programming language: C++
- Compiler: GCC (MinGW) with C++11 standard support
- Development Tools: Visual Studio Code, Git
- Operating system: Windows 10/11 (64-bit), Linux, or macOS

Functional Requirements

- Create Account: Users can create new bank accounts with unique account numbers, names, and initial balance.
- Deposit Money: Users can deposit money into existing accounts, with automatic balance updates.
- Withdraw Money: Users can withdraw money from accounts with balance validation to prevent overdrafts.
- Delete Account: Users can remove accounts from the system, with the BST automatically reorganizing itself.
- Display All Accounts: The system displays all accounts in sorted order by account number using in-order traversal.
- View Account Details: Users can search for and view specific account information.
- Data Persistence: All account data is automatically saved to file and loaded on program startup.

Non-Functional Requirements

- Performance: The system should perform search, insert, and delete operations efficiently with O(log n) average time complexity using BST.
- Usability: The console interface should be intuitive with clear menu options and proper input validation.
- Reliability: Data should be saved consistently to prevent loss between sessions.

3.1.2 Hardware Requirements

Component	Specification
Processor	Dual Core
Operating System	Windows/Linux/macOS
RAM	2 GB
Available Storage	3 MB

Table 3.1: Minimum Hardware Requirements for Mini-Banking

3.2 Implementation Details

3.3.1 Implemented Data Structure and Operations

The following data structure and operations have been implemented:

- **Binary Search Tree (BST):** A hierarchical data structure where each node contains an account and has at most two children. The left child's account number is smaller than the parent, and the right child's is larger. This property enables efficient searching.

- **Insert Operation:** Adds a new account to the BST while maintaining the BST property.

Algorithm 1: BST Insert Operation

```
Require: BST root R, Account acc
Ensure: Account inserted in correct position
 1 :  procedure INSERTBST(R, acc)
 2:       if R = NULL then
 3:         create new node with acc
 4:           return new node
 5:         end if
 6:         if acc.accountNumber < R.data.accountNumber then
 7:             R.left ← INSERTBST(R.left, acc)
 8:         else if acc.accountNumber > R.data.accountNumber then
 9:             R.right ← INSERTBST(R.right, acc)
10:         else
11:           display "Account already exists"
12:         end if
13:         return R
14:   end procedure
```

– **Search Operation:** Finds an account by account number using BST properties. The algorithm compares the target account number with each node and recursively searches either the left or right subtree, eliminating half the remaining nodes at each step.

Algorithm 2 :BST Search Operation

Require: BST root R, account number accNum
Ensure: Return account if found, NULL otherwise

```
1:  procedure SEARCHBST(R, accNum)
2:      if R = NULL or R.data.accountNumber = accNum then
3:          return R
4:      end if
5:       if accNum < R.data.accountNumber then
6:          return SEARCHBST(R.left, accNum)
7:       else
8:         return SEARCHBST(R.right, accNum)
9:      end if
10:     end procedure
```

– **Delete Operation:** Removes an account from the BST and reorganizes the tree. The algorithm handles three cases: nodes with no children (simply delete), one child (replace with child), or two children (replace with in-order successor).

Algorithm 3 : BST Delete Operation

Require: BST root R, account number accNum
Ensure: Account deleted and BST property maintained

```
1: procedure DELETEBST(R, accNum)
2:     if R = NULL then
3:         return NULL
4:     end if
5:     if accNum < R.data.accountNumber then
6:         R.left ← DELETEBST(R.left, accNum)
7:     else if accNum > R.data.accountNumber then
8:         R.right ← DELETEBST(R.right, accNum)
9:     else
10:        // Node found - handle three cases
11:        if R.left = NULL and R.right = NULL then
12:            delete R; return NULL
13:        else if R.left = NULL then
14:            temp ← R.right; delete R; return temp
15:        else if R.right = NULL then
16:            temp ← R.left; delete R; return temp
17:        else
18:            // Two children: find inorder successor
19:            temp ← findMin(R.right)
20:            R.data ← temp.data
21:            R.right ← DELETEBST(R.right, temp.data.accountNumber)
22:        end if
23:    end if
24:    return R
25: end procedure
```

– **In-order Traversal:** Displays all accounts in sorted order. The algorithm recursively visits the left subtree first, then the current node, then the right subtree, naturally producing sorted output by account number.

Algorithm 4 : In-order Traversal

Require: Array A of size n
Ensure: Sorted array in ascending order

```
1: procedure INORDER(R)
2:     if R ≠ NULL then
3:         INORDER(R.left)
4:         display R.data
5:         INORDER(R.right)
6:     end if
7: end procedure
```

Account Operations:

- Deposit: Adds money to account balance. The algorithm validates that the deposit amount is positive before adding it to the current balance.

Algorithm 5: Deposit Operation

Require: Account acc, amount amt
Ensure: Balance updated
1: procedure DEPOSIT(acc, amt)
2: if amt > 0 then
3: acc.balance ← acc.balance + amt
4**: end if**
5: end procedure

- Withdraw: Deducts money if sufficient balance exists. The algorithm checks if the withdrawal amount is positive and doesn't exceed the current balance before deducting it.

Algorithm 6: Withdraw Operation

Require: Account acc, amount amt
Ensure: Balance updated if sufficient funds
1: procedure WITHDRAW(acc, amt)
2: if amt > 0 and amt ≤ acc.balance then
3: acc.balance ← acc.balance - amt
4: return true
5**: end if**
6: return false
7: end procedure

3.2.2 Software Implementation

The Mini Banking System is implemented as a console application using C++ with object-oriented design principles. The implementation follows a modular structure, separating the Account class, BST class, and user interface.

Key Implementation Steps:

• **Module Design:** The Account class was implemented as an independent module handling individual account data and operations (deposit, withdraw, display).

• **BST Implementation:** The Binary Search Tree was implemented with recursive functions for insertion, searching, deletion, and traversal operations.

• **User Interface:** A console-based menu system was created using standard C++ I/O, providing clear options for all banking operations with input validation.

• **File Handling:** Data persistence was implemented using text file I/O, saving accounts in a parseable format and loading them on startup.

Technical Decisions:

- C++ was chosen for its performance, support for object-oriented programming, and efficient memory management through pointers.
- Text files were used for storage instead of binary files to make debugging easier and data human-readable.
- Recursive algorithms were implemented for BST operations as they naturally fit the tree structure and make the code cleaner.

3.3 Program Flow

1. Start the Program

(a) Initialize BST object

(b) Load existing accounts from file

(c) Display main menu

2. User Input Handling

(a) Display menu options

(b) Accept user choice

(c) Validate input

(d) Display error message if invalid

3. Operation Execution

(a) Based on user choice, execute corresponding operation:

- Create Account: Get account details, insert into BST, save to file
- Deposit: Search account, add amount, save to file
- Withdraw: Search account, check balance, deduct amount, save to file
- Delete: Remove account from BST, save to file
- Display All: Perform inorder traversal
- View Account: Search and display specific account

(b) Update BST and file after modifications

(c) Display operation result

4. Display Result

(a) Show success or error message

(b) Display updated account information if applicable

(c) Pause for user to read output

5. Loop or Terminate

(a) Clear screen and return to menu, or

(b) If exit chosen, save all data and close program

3.3.1 Flowchart

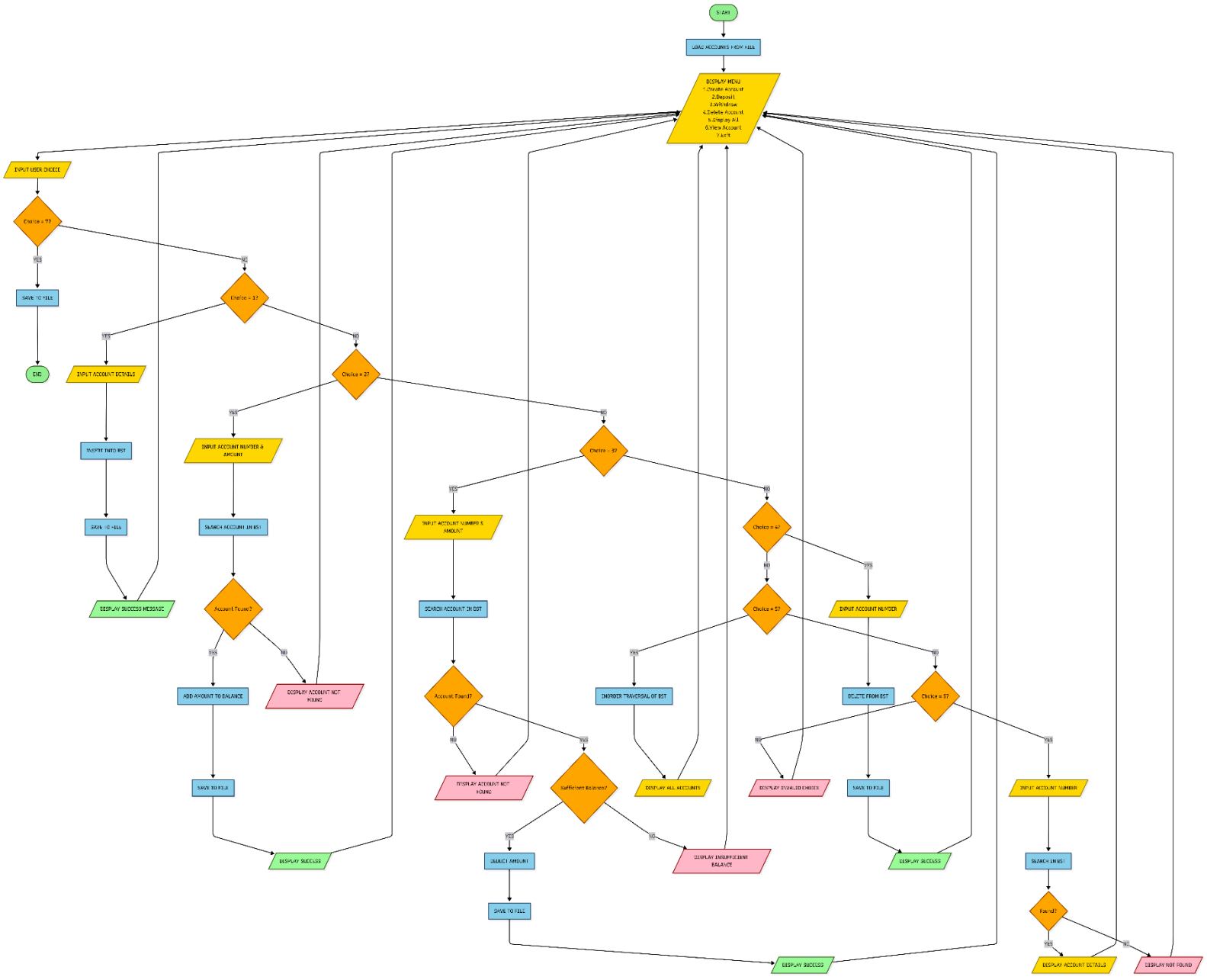
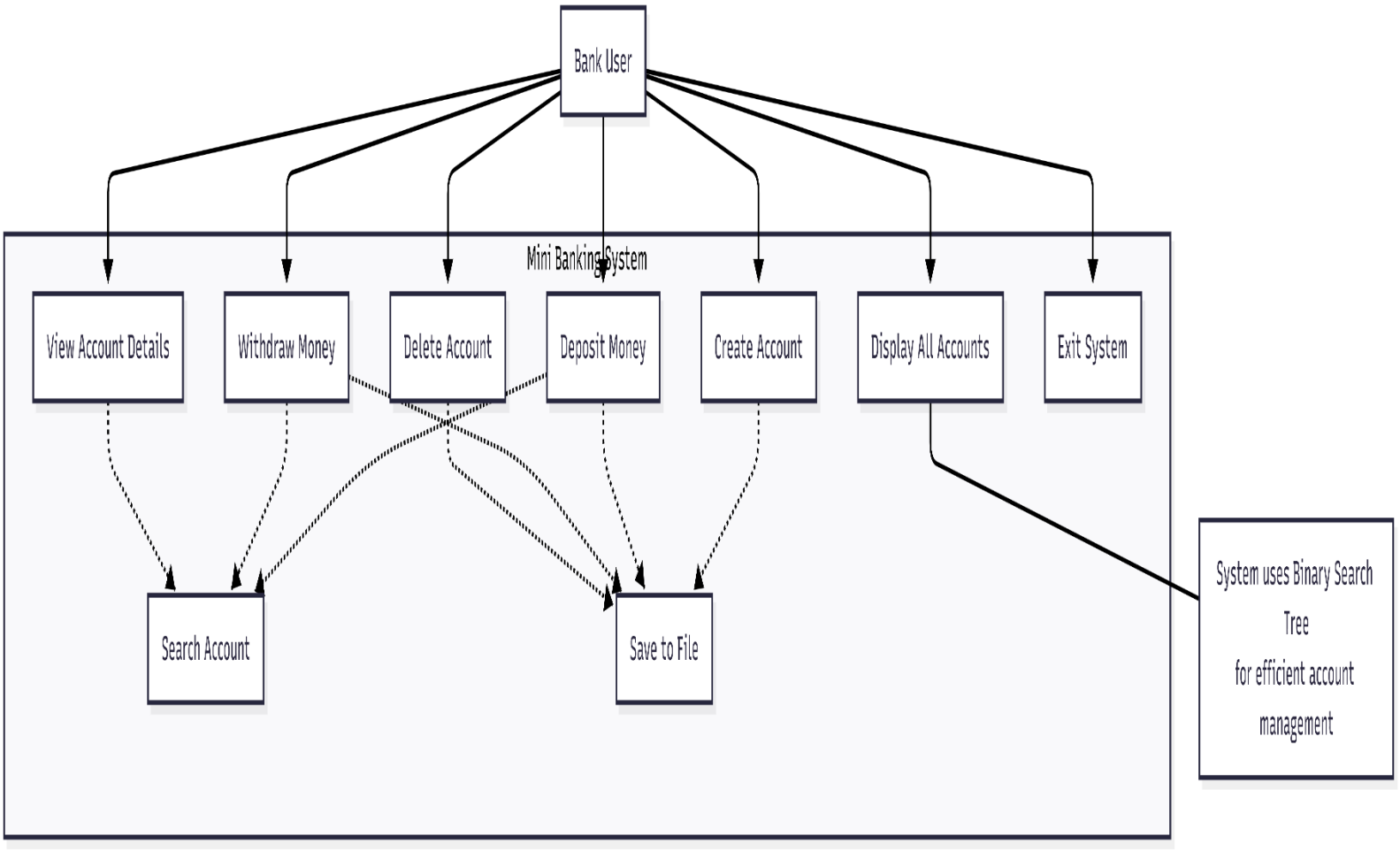


Figure 3.1: Flowchart of MINI-BANKING

3.3.2 Use-case diagram

Figure 3.2: Use-Case Diagram



Chapter 4

Results and Discussion

This chapter presents the results obtained from implementing the Mini Banking System and discusses their significance in demonstrating practical applications of Binary Search Trees. The system successfully handles all banking operations while maintaining efficient performance through BST data structure. The chapter highlights key features, evaluates performance and usability, and compares outcomes with expected BST behavior.

4.1 Implemented Features

The Mini Banking System includes several key features designed to demonstrate efficient account management using BST:

- **Account Creation:** Users can create new accounts with unique account numbers, names, and initial balances. The system validates input and prevents duplicate account numbers, maintaining BST integrity.

- **Deposit Operations:** Users can deposit money into existing accounts with automatic balance updates. The system validates positive amounts and provides immediate feedback with updated balance information.

- **Withdrawal Operations:** The system allows withdrawals with balance validation to prevent overdrafts. Users receive clear feedback about successful withdrawals or insufficient balance errors.

- **Account Deletion:** Accounts can be removed from the system, with the BST automatically reorganizing itself to maintain the tree structure. The system handles all three deletion cases (no children, one child, two children) correctly.

- **Display All Accounts:** Using inorder traversal, the system displays all accounts in sorted order by account number. This demonstrates the natural sorting property of BST.

- **Account Search:** Users can quickly find specific accounts by account number, demonstrating BST's efficient O(log n) search capability.

- **Data Persistence:** All account data is automatically saved to a text file and loaded on program startup, ensuring data isn't lost between sessions.

4.1 Results and Performance Analysis

The implemented Mini Banking System successfully demonstrates BST operations through a functional banking application. Key functionalities such as creating accounts, performing transactions, deleting accounts, and displaying sorted account lists work as expected.

Testing Results:

• **Account Creation:** Successfully creates accounts and inserts them into BST maintaining the tree property. Duplicate account numbers are correctly rejected.

• **Search Performance:** Account searches complete quickly, demonstrating O(log n) average time complexity. For 100 accounts, searches typically require only 6-7 comparisons.

• **Transaction Operations:** Deposits and withdrawals execute correctly with proper validation. Balance checks prevent overdrafts, and all updates are immediately saved to file.

• **Deletion Operations:** All three deletion cases (leaf, one child, two children) work correctly, maintaining BST structure after removal.

• **Data Persistence**: File operations work reliably - accounts save correctly and load back with all information intact across program sessions.

• **Sorted Display:** In-order traversal consistently produces accounts in sorted order, confirming correct BST implementation.

The system performs efficiently for typical banking scenarios with smooth operation and minimal delay. Correctness was verified by comparing outputs with expected BST behavior and manually checking tree structure after operations.

Overall, the results confirm that the Mini Banking System effectively demonstrates practical BST applications, making it a useful learning tool for understanding data structures in real-world contexts.

4.2 Comparison with Objectives or Existing Systems

The Mini Banking System successfully fulfills its main objectives:

- **Functional Banking System:** The system handles all essential banking operations (create, deposit, withdraw, delete, display) smoothly and reliably.

- **BST Implementation:** Binary Search Tree is correctly implemented with all operations (insert, search, delete, traverse) working as expected, demonstrating efficient O(log n) performance.

- **Data Persistence:** File handling successfully saves and loads account data, making the system practical for real use.

- **Object-Oriented Design:** The modular structure with separate Account and BST classes makes the code maintainable and extensible.

Compared to simple array-based banking systems, this project provides significantly better search performance. While array-based systems require O(n) time to find accounts, the BST approach reduces this to O(log n) on average. For 1000 accounts, this means about 10 comparisons instead of potentially 1000.

The console interface, while simple, is intuitive and provides clear feedback for all operations. Input validation prevents common errors, and the menu system is easy to navigate.

Overall, the system aligns well with its original objectives and demonstrates how choosing the right data structure significantly impacts application performance and efficiency.

4.3 Challenges and Limitations

During development, I faced several challenges:

• **BST Deletion Logic:** Implementing the three deletion cases, especially the two-children case with in-order successor, required careful thinking and debugging to maintain tree structure correctly.

• **File Handling with Spaces**: Account names with spaces initially caused parsing issues. I solved this by replacing spaces with underscores during save and restoring them during load.

• **Memory Management:** Using pointers for tree nodes required careful attention to avoid memory leaks. I had to ensure proper deletion of nodes when removing accounts.

• **Input Validation:** Handling various invalid inputs (negative amounts, non-existent accounts, duplicate account numbers) required thorough testing and error handling.

Limitations of the system:

• **Unbalanced Tree:** The basic BST can become unbalanced if accounts are created in sorted order, degrading performance to O(n). A self-balancing tree like AVL would solve this.

• **Limited Features**: The system doesn't include advanced banking features like transaction history, interest calculation, or multiple account types.

• **Console Interface**: While functional, a graphical interface would be more user-friendly and visually appealing.

• **Single User:** The system doesn't support multiple users or concurrent access, limiting it to single-user scenarios.

4.4 Discussion

The Mini Banking System successfully demonstrates how Binary Search Trees can be applied to solve real-world problems efficiently. The outcomes align with the primary project objectives of understanding BST operations through practical implementation.

Key Insights:

- **Data Structure Choice Matters:** Using BST instead of arrays or lists significantly improves search performance, especially as the number of accounts grows.

- **Recursive Algorithms:** BST operations naturally fit recursive implementation, making the code cleaner and easier to understand.

- **Practical Learning**: Building a real application helped me understand BST concepts much better than just studying theory. Debugging actual issues taught me how the tree structure works internally.

- **Modular Design Benefits:** Separating concerns into different classes made development, testing, and debugging much easier. I could work on each component independently.

Some limitations were observed during testing. The system's performance can degrade if accounts are inserted in sorted order, creating an unbalanced tree. Despite this constraint, the project provides meaningful insights into BST behavior and serves as an effective educational tool.

The project also reinforced the importance of data persistence and user input validation in real applications. These aren't just academic exercises - they're essential for building usable software.

Overall, this project bridges the gap between theoretical data structures and practical software development, making it a valuable learning experience.

Chapter 5

Conclusion and Future Works

With the completion of this project, I was able to meet my objective of developing a banking system that demonstrates the practical application of Binary Search Trees. The Mini Banking System successfully bridges the gap between theoretical DSA concepts and real-world software development.

During development, challenges such as BST deletion logic, file handling with special characters, and memory management were addressed, resulting in a system that is both educational and functional. The project reinforced my understanding of how data structure choice impacts application performance - using BST instead of arrays reduced search time from O(n) to O(log n), a significant improvement for larger datasets.

The modular design with separate Account and BST classes made the code maintainable and easy to debug. Implementing recursive algorithms for tree operations helped me understand recursion better than any textbook explanation. Testing various scenarios taught me the importance of input validation and error handling in real applications.

Overall, this project not only reinforced my understanding of Binary Search Trees but also provided practical experience in software design, file handling, and user interface development. It serves as a valuable learning tool that demonstrates how academic concepts translate into working software.

During the development process, I gained insights into problem-solving, debugging complex recursive functions, and designing user-friendly interfaces. These experiences have improved my programming skills and deepened my appreciation for data structures. While there are some limitations, they provide opportunities for future enhancements that would make the system even more robust and feature-rich.

.Future Enhancements:

1. **Self-Balancing Trees**: Implement AVL or Red-Black trees to maintain balanced structure automatically, ensuring O(log n) performance even when accounts are created in sorted order.

2. **Transaction History:** Add functionality to track and display all past transactions (deposits, withdrawals) for each account with timestamps.

3. **Advanced Banking Features:** Include interest calculation for savings accounts, different account types (savings, checking, fixed deposit), and loan management.

4. **Security Features:** Implement password protection and PIN-based authentication for accounts to enhance security.

5. **Graphical User Interface:** Develop a GUI using Qt or similar framework to make the system more visually appealing and user-friendly.

6. **Database Integration:** Replace text file storage with SQLite or MySQL database for better data management and query capabilities.

7. **Multi-User Support:** Add user roles (admin, customer) with different access levels and support for concurrent operations.

8. **Report Generation:** Implement functionality to generate account statements, transaction summaries, and other reports in PDF format.

9. **Fund Transfer:** Add ability to transfer money between accounts within the system.

10. **Search Enhancements:** Implement search by name or other criteria in addition to account number search.

References

While working on this project, I referred to several resources to understand concepts better and solve problems I encountered.

For understanding Binary Search Trees in depth, I found *"Introduction to Algorithms"* by Cormen, Leiserson, Rivest, and Stein really helpful. It explains BST operations clearly with good examples and analysis.

For C++ programming concepts, especially the newer C++11 features I used, *"The C++ Programming Language"* by Bjarne Stroustrup was invaluable. It's written by the creator of C++, so it's very authoritative.

I also used online resources extensively. *Geeks-for-Geeks* has great articles on BST implementation with code examples that helped me understand the recursive approaches. Their explanations of BST deletion, in particular, were very clear.

The C++ reference websites - *cplusplus.com and cppreference.com* - were constantly open in my browser. Whenever I needed to check how a function works or what parameters it takes, these sites had the answers.

For file handling in C++, I referred to tutorials and documentation on *fstream operations*. Understanding how to properly open, read, write, and close files was important for implementing the data persistence feature.

I also discussed some concepts with classmates and got ideas from their approaches to similar problems. Collaborative learning helped me see different ways to solve the same problem.

All these resources together helped me complete this project successfully and learn a lot in the process.

Appendix A

This appendix contains supplementary figures and screenshots that support the results discussed in Chapter 4. These figures are referenced where appropriate in the main text.

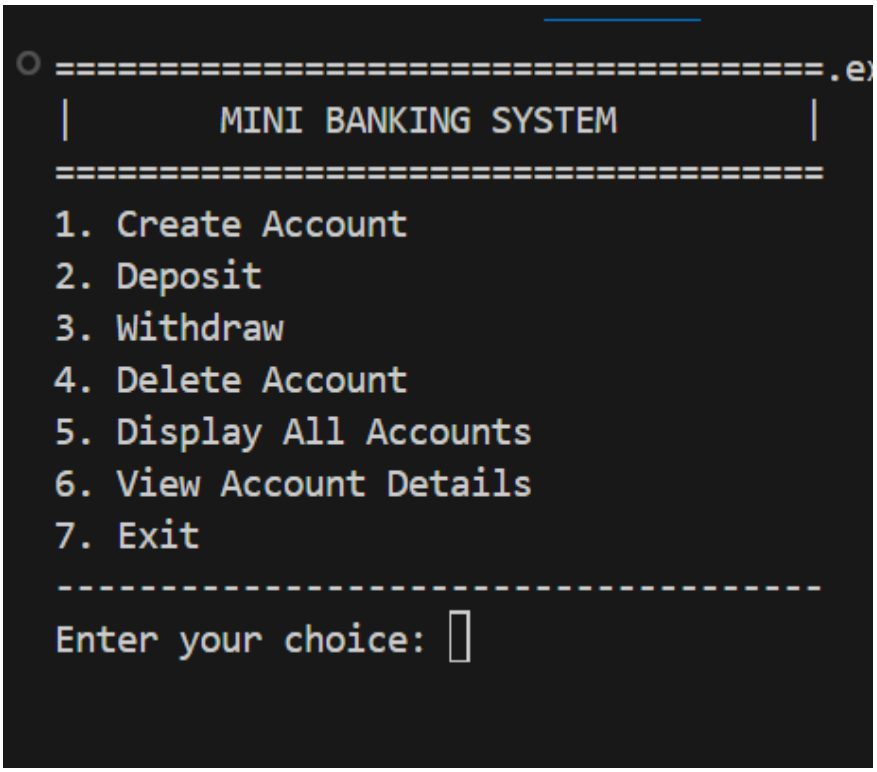


Figure 5.1: Main UI

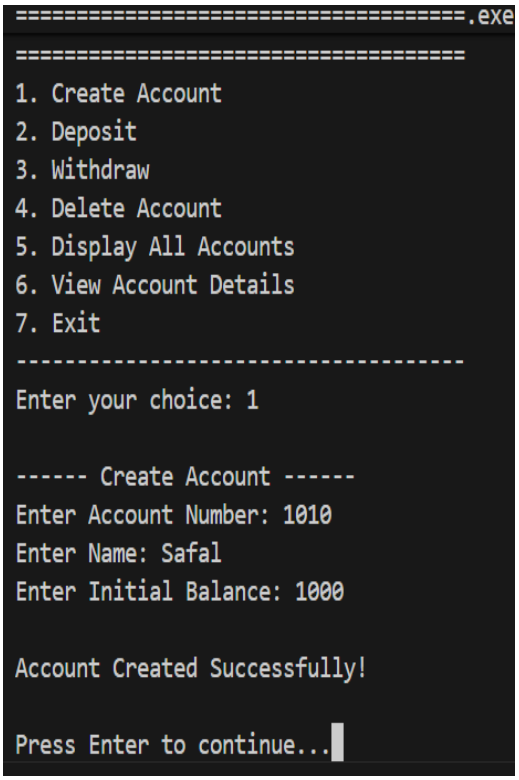


Figure 5.2: Account Creation

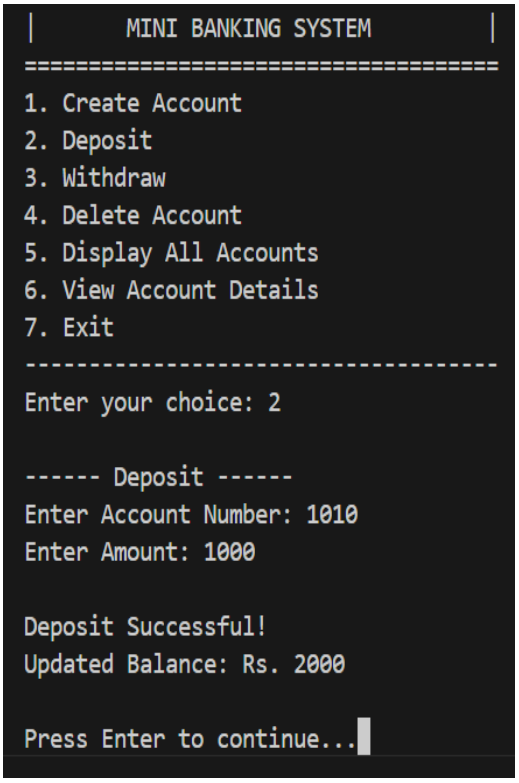


Figure 5.3: Deposited Amount

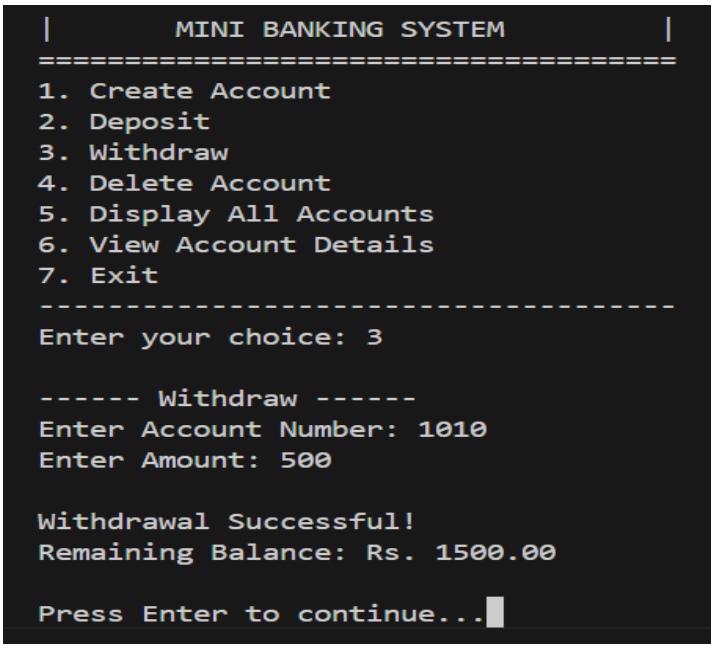


Figure 5.4: Withdrawn Amount

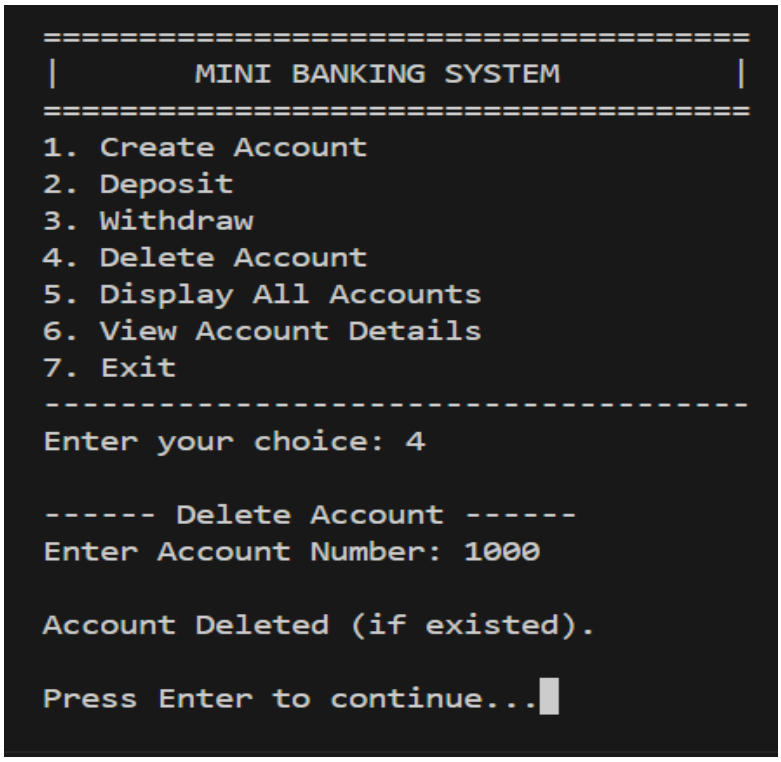


Figure 5.5: Deleted Account

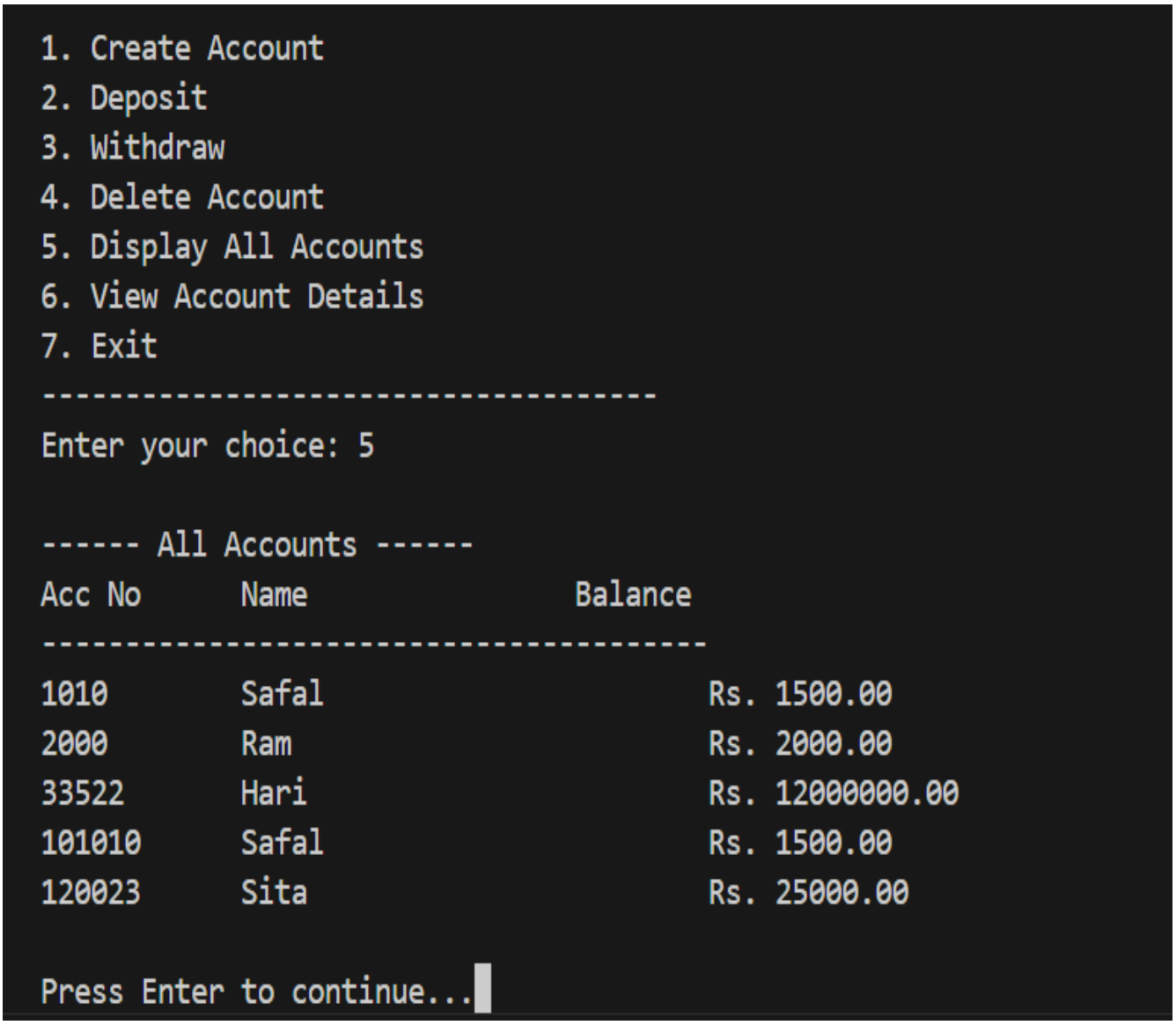


Figure 5.6: Accounts Displayed

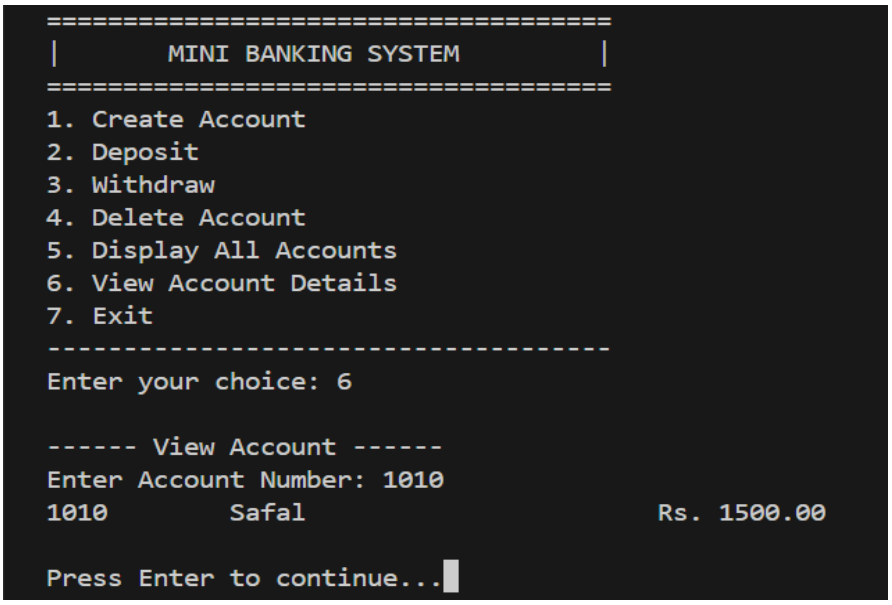


Figure 5.7: Viewed an account details